ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Subtyping — Class Challenge Set, Solutions

Challenge 1

Recall the relevant rules:

- **Record width:** more fields → subtype.
- **Record depth:** componentwise subtyping on fields.
- **Functions:** $A \to B <: C \to D$ iff $C <: A$ (contravariant input) and $B <: D$ (covariant output).
- **Bottom:** `Nothing <: T` for every T.

S	T	$S \; ? \; T$	**Justification**
$\{x : \mathsf{Int}, y : \mathsf{Int}\}$	$\{x : \mathsf{Int}\}$	$S <: T$	Width: S has all fields of T (and more).
$\{p : \mathsf{Int} \to \mathsf{Int}\}$	$\{p : \mathsf{Int} \to \mathsf{Int}, q : \mathsf{Bool}\}$	$T <: S$	Width: T has all fields of S (and more).
$\mathsf{Int} \to \mathsf{Int}$	$\mathsf{Int} \to \mathsf{Any}$	$S <: T$	Same input; output $\mathsf{Int} <: \mathsf{Any}$ (covariant).
$\mathsf{Any} \to (\mathsf{Any} \to \mathsf{Int})$	$\mathsf{Any} \to (\mathsf{Int} \to \mathsf{Int})$	$S <: T$	Inputs equal; on outputs need $\mathsf{Any} \to \mathsf{Int} <: \mathsf{Int} \to \mathsf{Int}$, which holds: $\mathsf{Int} <: \mathsf{Any}$ (contravariant input) and $\mathsf{Int} <: \mathsf{Int}$.
$(\mathsf{Any} \to \mathsf{Any}) \to \mathsf{Int}$	$(\mathsf{Any} \to \mathsf{Int}) \to \mathsf{Int}$	$S <: T$	Output equal; on inputs (contravariant) need $\mathsf{Any} \to \mathsf{Int} <: \mathsf{Any} \to \mathsf{Any}$, which holds by output covariance.
$\{f : \mathsf{Animal} \to \mathsf{Int}\}$	$\{f : \mathsf{Cat} \to \mathsf{Int}\}$	$S <: T$	Depth with function contravariance: $\mathsf{Cat} <: \mathsf{Animal}$ gives $\mathsf{Animal} \to \mathsf{Int} <: \mathsf{Cat} \to \mathsf{Int}$.
$\{g : \mathsf{Int} \to \mathsf{Int}\}$	$\{g : \mathsf{Int} \to \mathsf{Nothing}\}$	$T <: S$	$S \not<: T$ because $\mathsf{Int} \not<: \mathsf{Nothing}$. Conversely $\mathsf{Nothing} <: \mathsf{Int}$ holds, so $T <: S$.

Challenge 2

`apply(x: Int, f: Int => Int): Int` needs `f` of type $\mathsf{Int} \to \mathsf{Int}$. For the actual argument of type $A \to B$, we need $A \to B \ <: \ \mathsf{Int} \to \mathsf{Int}$, i.e. $\mathsf{Int} <: A$ and $B <: \mathsf{Int}$.

Application	Checks?	Direction at play
`apply(5, (x: Int) => x + 1)`	✓	Trivially $\mathsf{Int} \to \mathsf{Int} \ <: \ \mathsf{Int} \to \mathsf{Int}$.
`apply(5, (x: Any) => x.hashCode)`	✓	Contravariance of input: $\mathsf{Int} <: \mathsf{Any}$, output $\mathsf{Int} <: \mathsf{Int}$. A function that accepts *anything* certainly accepts an `Int`.
`apply(5, (x: Int) => x.toString)`	✗	Output covariance fails: $\mathsf{String} \not<: \mathsf{Int}$.
`apply(5, (x: Nothing) => x)`	✗	Input contravariance fails: $\mathsf{Int} \not<: \mathsf{Nothing}$. There is no actual `Int` we could pass to a function that only accepts `Nothing`.

Challenge 3

A first version of the trait:

```
trait Container[A]:
  // abstract
  def add(x: A): Container[A]
  def toList: List[A]
  def fromList(xs: List[A]): Container[A]

  // concrete
  def size: Int                                  = toList.size
  def isEmpty: Boolean                           = toList.isEmpty
  def contains(x: A): Boolean                    = toList.contains(x)
  def concat(other: Container[A]): Container[A] = fromList(toList ++ other.toList)
```

Adding `map`. Signature: `def map[B](f: A => B): Container[B]`. The current abstract surface is *not* expressive enough: `fromList: List[A] => Container[A]` produces a `Container[A]`, not a `Container[B]`. We must upgrade `fromList` to be polymorphic in the element type — it becomes a *factory*:

```
trait Container[A]:
  def add(x: A): Container[A]
  def toList: List[A]
  def fromList[B](xs: List[B]): Container[B]    // upgraded

  def map[B](f: A => B): Container[B] = fromList(toList.map(f))
  // size, isEmpty, contains, concat as before
```

A concrete implementation:

```
class ListContainer[A](items: List[A]) extends Container[A]:
  def add(x: A): Container[A]                  = ListContainer(x :: items)
  def toList: List[A]                          = items
  def fromList[B](xs: List[B]): Container[B]   = ListContainer(xs)
```

Key decisions.

- *Price of routing everything through* `toList`. Every concrete operation is forced to materialise the whole list: `size` is $O(n)$ even when the implementation could maintain a cached counter; `contains` is a linear scan even on a tree-backed implementation that could do $O(\log n)$. The fix is to expose more abstract methods (`size`, `contains`, …) and provide `toList`-based defaults only as fallback — implementers override them with their own efficient versions.
- *`concat` without `toList`.* Add an abstract `addAll(xs: List[A]): Container[A]` (or accept a whole `Container[A]` directly, e.g. `merge(other: Container[A]): Container[A]`). Then `concat = (this.addAll _).compose(_.toList)` — or, even better, leave `concat` abstract and let each implementation merge its own internal representation.

Challenge 4

(a) Design 2 (parametric). With

```
trait Equatable[A]:
  def equalsTo(that: A): Boolean
```

`Person extends Equatable[Person]` constrains `equalsTo` to take only a `Person`. Comparing `Person` to `Car` is rejected statically. Design 1 (subtyping) is too loose: any `Equatable` can be compared with any other, so `person.equalsTo(car)` typechecks, which is exactly the nonsense we want to forbid.

(b) Try:

```
trait Equatable[A]:
  def compareTo(that: A): Int
  def < (that: A): Boolean = compareTo(that) <  0
  def <=(that: A): Boolean = compareTo(that) <= 0
  def >=(that: A): Boolean = compareTo(that) >= 0
```

This compiles. But the design has two latent problems that surface as soon as you push it further:

1. **No constraint that `A` is "the implementing type".** Nothing prevents `class Person extends Equatable[Car]`, which is meaningless. The compiler is happy because there is no link between `A` and the class that implements the trait.
2. **Methods that need to *return* an `A` cannot be written.** Try adding a `max`:

   ```
   def max(that: A): A = if compareTo(that) >= 0 then this else that
   ```

   This **does not compile**: inside the trait `this : Equatable[A]`, not `A`, so `this` cannot be returned where `A` is expected.

The fix is **F-bounded polymorphism** combined with a self-type:

```
trait Equatable[A <: Equatable[A]]:
  this: A =>
  def compareTo(that: A): Int
  def < (that: A): Boolean = compareTo(that) <  0
  def max(that: A): A      = if compareTo(that) >= 0 then this else that
```

The bound `A <: Equatable[A]` says "the only sensible instantiations are `Person extends Equatable[Person]`, `Money extends Equatable[Money]`, …". The self-type `this: A =>` tells the compiler that, inside the trait, `this` may be used as a value of type `A` — which is exactly what enables `max` to return `this`.

The takeaway. Subtyping alone (Design 1) is too permissive: it admits cross-type comparisons. Parametricity alone (Design 2) is too disconnected: it admits arbitrary mismatches between the type parameter and the implementing class. Robust generic interfaces that need to *talk about the implementing type* require the two mechanisms to be coupled by F-bounded polymorphism.

Challenge 5

Expression	Result	Determining rule
`Reminder("buy milk", "Mon").header`	`"Reminder for Mon"`	Dynamic dispatch on `header` → `Reminder`'s override.
`Reminder("buy milk", "Mon").render`	`"[Reminder for Mon]\nbuy milk"`	`render` inherited from `Note` (not overridden in `Reminder`). Inside it, `header` is called on `this`; dynamic dispatch picks `Reminder.header`.
`Quote("less is more", "Mies").render`	`"\"less is more\"\n  — Mies"`	Dynamic dispatch on `render` → `Quote`'s override; `header` is not consulted at all.
`PlainNote("hello").render`	`"[Note]\nhello"`	Both `render` and `header` inherited as defaults from `Note`.
`new Note { val text = "x" }.header`	`"Note"` (default)	Anonymous subclass of a `sealed` trait — compiles *iff the expression sits in the same file as* `Note`; otherwise the seal rejects it. `header` is the

Expression	Result	Determining rule
		default from `Note`.

Food for further thinking.

- *Does the last one compile?* Only inside the file where `Note` is declared. `sealed` confines direct subclasses (including anonymous ones) to the defining compilation unit. The expression also has to supply the abstract member `text`, which it does.
- *Tracing the second expression carefully.* `r.render` enters `Note.render`'s body, which contains `s"[$header]\n$text"`. The receiver of `header` and `text` is the implicit `this`, whose runtime class is `Reminder`. The default `Note.header` is *shadowed* by `Reminder.header`; the literal `text` is provided by `Reminder`'s constructor. The static dispatch is on the method `render` (resolved to `Note.render` at compile time of the inherited member), but every call *inside* that body is virtual.

Challenge 6

`Reminder("buy milk", "Mon").combinedWith(Quote("less is more", "Mies"))` produces:

```
[Reminder for Mon]
buy milk
---
"less is more"
  — Mies
```

Trace of the two `render` calls.

- `this.render` — `this : Reminder`. `Reminder` does not override `render`, so the inherited `Note.render` executes; inside it `header` resolves dynamically to `Reminder.header` → `"Reminder for Mon"`. Output: `[Reminder for Mon]\nbuy milk`.
- `other.render` — `other` is declared `Note` but its runtime class is `Quote`. Dynamic dispatch selects `Quote.render`, which does not even touch `header`. Output: `"less is more"\n  — Mies`.

Why both must be dynamic. `combinedWith` is written in terms of the static interface `Note`, but the substitution principle (LSP) requires every `Note` instance to behave according to *its actual class*. If `other.render` were resolved statically to `Note.render`, a `Quote` passed where a `Note` is expected would silently lose its custom rendering — the very point of extending and overriding would be defeated. The same argument applies symmetrically to `this.render`: dynamic dispatch is what makes "program against `Note`" mean "actually get the behaviour the runtime subclass defines".